PYTHON PROGRAMS — EXPLAINED

Updated edition: Python only. No C-language comparisons. The aim is to understand what each Python statement does and why it is used.

How to study: Read the program once, then read the explanation in order. Trace the values on paper, run the program, and finally try writing it without looking.

1. FIND-S Algorithm

Overall purpose: Finds the most specific hypothesis that covers all positive training examples.

- `import csv` loads Python's CSV module so rows can be read from a spreadsheet-style file.
- `with open(...) as file` opens the CSV file and closes it automatically when the block ends.
- `csv.reader(file)` reads the file row by row; `list(...)` stores all rows in memory.
- `data[0][:-1]` takes the header row except its last item, because the last column is the target.
- `data = data[1:]` removes the header, leaving only training examples.
- `['0'] * len(attributes)` creates one initial placeholder for every attribute. Here `0` means no value has been learned yet.
- `for row in data` processes each training example in sequence.
- `row[-1]` accesses the last item of the current row, the target label.
- `strip().lower()` removes surrounding spaces and makes the label lowercase before checking it.
- `if target == 'yes'` means only positive examples update FIND-S.
- `range(len(attributes))` produces the index positions of all attributes.
- If the hypothesis position is `0`, the first positive example supplies its value.
- If the learned value differs from the new positive example, `?` is used to mean any value is accepted.
- `print(...)` displays the final learned hypothesis.

Important: Your CSV must have a header row, attributes first, and the target class in the last column.

Code:

```
import csv

with open("training_data.csv", "r") as file:
    data = list(csv.reader(file))

attributes = data[0][:-1]
data = data[1:]
hypothesis = ["0"] * len(attributes)

for row in data:
    if row[-1].strip().lower() == "yes":
        for i in range(len(attributes)):
            if hypothesis[i] == "0":
                hypothesis[i] = row[i]
            elif hypothesis[i] != row[i]:
                hypothesis[i] = "?"

print("Final Hypothesis:", hypothesis)
```

2. Candidate Elimination

Overall purpose: Maintains a specific boundary S and a general boundary G while processing positive and negative examples.

- `S = ['0'] * n` starts the specific boundary with the most specific placeholder for every attribute.
- `G = [['?'] * n]` starts the general boundary with one rule that accepts every possible attribute value.
- `x = row[:-1]` extracts the attribute values from the current example; the last value is its class label.
- For a positive example, S is minimally generalized: an unknown position takes the example's value, and a conflicting position becomes `?`.

- `G = [...]` keeps only general hypotheses that are consistent with the positive example.
- For a negative example, a general hypothesis that already rejects the example can remain unchanged.
- If a general hypothesis covers a negative example, the program creates specializations by replacing a `?` with a value from S that excludes the negative example.
- `copy()` creates a separate list so changing one specialization does not change the original rule.
- `append()` adds each valid specialization to the new general boundary.
- The final `print` statements show S and G after all examples have been processed.

Important: This is a compact teaching implementation. Candidate Elimination implementations can differ in how they remove duplicate or overly general/specific hypotheses; compare the resulting boundaries with your lab's expected output.

Code:

```
import csv

with open("training_lab2.csv", "r") as file:
    data = list(csv.reader(file))

attributes = data[0][:-1]
data = data[1:]
n = len(attributes)

S = ["0"] * n
G = [["?"] * n]

for row in data:
    x = row[:-1]
    target = row[-1].strip().lower()

    if target == "yes":
        for i in range(n):
            if S[i] == "0":
                S[i] = x[i]
            elif S[i] != x[i]:
                S[i] = "?"

        G = [g for g in G if all(
            g[i] == "?" or g[i] == x[i] for i in range(n)
        )]

    elif target == "no":
        new_G = []
        for g in G:
            if all(g[i] == "?" or g[i] == x[i] for i in range(n)):
                for i in range(n):
                    if g[i] == "?":
                        if S[i] != "?" and S[i] != "0" and S[i] != x[i]:
                            new_g = g.copy()
                            new_g[i] = S[i]
                            new_G.append(new_g)
            else:
                new_G.append(g)
        G = new_G

print("Specific Boundary (S):", S)
print("General Boundary (G):", G)
```

3. ID3 Decision Tree

Overall purpose: Builds a decision tree by choosing the attribute with the highest information gain at each step.

- `math` provides `log2`, which is used in entropy; `Counter` counts how many times each class occurs.
- `csv.DictReader` turns each CSV row into a dictionary, so values can be accessed by column name.
- `TARGET` stores the name of the class column; `ATTRIBUTES` contains all other column names.
- `Counter(row[TARGET] for row in rows)` counts the target labels in the current subset.
- Entropy is `-sum(p * log2(p))`: it is zero when all rows have the same class and larger when classes are mixed.
- Information gain is the original entropy minus the weighted entropy after splitting on an attribute.
- `set(...)` gets the distinct values of an attribute.

- `subset` contains only rows whose attribute has the current value.
- `max(attributes, key=...)` selects the attribute with the largest information gain.
- `tree = {best: {}}` creates a dictionary node; each branch will be stored under its attribute value.
- The recursive call `id3(subset, remaining)` builds the next level using the smaller subset and remaining attributes.
- `predict` follows the dictionary branches using the sample's values until it reaches a class label.

Important: The target column must be named PlayTennis. The sample dictionary must use the same attribute names and valid values as the CSV.

Code:

```
import csv
import math
from collections import Counter

with open("play_tennis_lab3.csv", "r") as file:
    data = list(csv.DictReader(file))

TARGET = "PlayTennis"
ATTRIBUTES = [a for a in data[0] if a != TARGET]

def entropy(rows):
    counts = Counter(row[TARGET] for row in rows)
    total = len(rows)
    return -sum((count / total) * math.log2(count / total)
                for count in counts.values())

def information_gain(rows, attribute):
    total_entropy = entropy(rows)
    values = set(row[attribute] for row in rows)
    weighted_entropy = 0

    for value in values:
        subset = [row for row in rows if row[attribute] == value]
        weighted_entropy += (len(subset) / len(rows)) * entropy(subset)

    return total_entropy - weighted_entropy

def id3(rows, attributes):
    labels = [row[TARGET] for row in rows]

    if len(set(labels)) == 1:
        return labels[0]
    if not attributes:
        return Counter(labels).most_common(1)[0][0]

    best = max(attributes, key=lambda a: information_gain(rows, a))
    tree = {best: {}}

    for value in set(row[best] for row in rows):
        subset = [row for row in rows if row[best] == value]
        remaining = [a for a in attributes if a != best]
        tree[best][value] = id3(subset, remaining)

    return tree

def predict(tree, sample):
    if not isinstance(tree, dict):
        return tree
    attribute = next(iter(tree))
    value = sample.get(attribute)
    if value not in tree[attribute]:
        return None
    return predict(tree[attribute][value], sample)

tree = id3(data, ATTRIBUTES)
sample = {"Outlook": "Sunny", "Temperature": "Cool",
          "Humidity": "High", "Wind": "Strong"}

print("Decision Tree:", tree)
print("Prediction:", predict(tree, sample))
```

4. Naive Bayes — Weather CSV

Overall purpose: Predicts a class using Bayes' rule and categorical feature values, with Laplace smoothing.

- `DictReader` reads each training row as a dictionary.

- `FEATURES` collects every column except the target column.
- `Counter` counts how many training rows belong to each class.
- `predict(sample)` receives a dictionary containing the feature values of one test example.
- The starting score `class_count / total` is the prior probability of the class.
- `value_count` counts training rows where both the class and the feature value match.
- `possible_values` counts distinct values for that feature; adding it in the denominator performs Laplace smoothing.
- `score *= ...` multiplies the conditional probability into the class score.
- `max(scores, key=scores.get)` returns the class with the largest calculated score.
- `DictReader` on the test file reads test examples; the loop predicts each row.

Important: Training and test CSV files must use matching feature names. The target column is Play.

Code:

```
import csv
from collections import Counter

with open("training_lab4.csv", "r") as file:
    train = list(csv.DictReader(file))

TARGET = "Play"
FEATURES = [key for key in train[0] if key != TARGET]
classes = Counter(row[TARGET] for row in train)
total = len(train)

def predict(sample):
    scores = {}

    for cls, class_count in classes.items():
        score = class_count / total

        for feature in FEATURES:
            value = sample[feature]
            value_count = sum(
                1 for row in train
                if row[TARGET] == cls and row[feature] == value
            )
            possible_values = len(set(row[feature] for row in train))
            score *= (value_count + 1) / (class_count + possible_values)

        scores[cls] = score

    return max(scores, key=scores.get)

with open("test_lab4.csv", "r") as file:
    tests = list(csv.DictReader(file))

for sample in tests:
    actual = sample.get(TARGET, "Unknown")
    print("Actual:", actual, "Predicted:", predict(sample))
```

5. Naive Bayes — Text Classification

Overall purpose: Classifies a sentence into a category by learning word frequencies for each category.

- `re` provides regular expressions; `Counter` stores word frequencies; `defaultdict(Counter)` creates a counter automatically for each class.
- `documents` is a list of pairs: the document text and its category label.
- `tokenize` converts text to lowercase words and ignores punctuation and digits.
- For every document, the program increments its class count and each word's count for that class.
- `vocabulary` is the set of all distinct words found in the training documents.
- `math.log` is used because multiplying many small probabilities can underflow; adding log probabilities is numerically safer.
- The class prior is the fraction of documents belonging to that category.
- `(word_count + 1) / (total_words + vocabulary_size)` is the Laplace-smoothed probability of a word.
- `max(scores, key=scores.get)` selects the category with the highest score.

Important: These sample training documents are built into the program. Replace or extend the list with the documents required by your lab.

Code:

```
import re
import math
from collections import Counter, defaultdict

documents = [
    ("team won cricket match", "Sports"),
    ("players scored goal in match", "Sports"),
    ("government passed new policy", "Politics"),
    ("minister announced policy", "Politics"),
    ("team scored goal", "Sports"),
    ("government discussed policy", "Politics")
]

word_counts = defaultdict(Counter)
class_counts = Counter()
vocabulary = set()

def tokenize(text):
    return re.findall(r"\b[a-z]+\b", text.lower())

for text, label in documents:
    class_counts[label] += 1
    for word in tokenize(text):
        word_counts[label][word] += 1
        vocabulary.add(word)

def predict(text):
    words = tokenize(text)
    total_docs = sum(class_counts.values())
    scores = {}

    for label in class_counts:
        score = math.log(class_counts[label] / total_docs)
        total_words = sum(word_counts[label].values())

        for word in words:
            probability = (word_counts[label][word] + 1) / (
                total_words + len(vocabulary)
            )
            score += math.log(probability)

        scores[label] = score

    return max(scores, key=scores.get)

sample = "team won cricket match"
print("Document:", sample)
print("Predicted Category:", predict(sample))
```

6. Naive Bayes — Heart Dataset

Overall purpose: Uses Gaussian Naive Bayes to classify numerical patient features.

- `defaultdict(list)` groups all rows by their target class.
- `float(...)` converts a CSV text value into a number.
- `mean` calculates the average of a feature's values for one class.
- `variance` measures how spread out those values are around the average.
- `gaussian` calculates the probability density of a value under a normal distribution.
- `max(var, 1e-9)` prevents division by zero when all values are identical.
- `math.log` converts products of probabilities into sums.
- For each class, the program starts with the class prior and adds the log Gaussian likelihood for every feature.
- The class with the highest score becomes the prediction.

Important: This Gaussian version expects every feature column to be numeric. If the dataset has text categories such as Male or High, this exact program needs categorical handling or numeric encoding before it can run.

Code:

```
import csv
import math
from collections import defaultdict

FILE = "heart.csv"
TARGET = "HeartDisease"

with open(FILE, "r") as file:
    data = list(csv.DictReader(file))

FEATURES = [key for key in data[0] if key != TARGET]
classes = defaultdict(list)

for row in data:
    classes[row[TARGET]].append(row)

def mean(values):
    return sum(values) / len(values)

def variance(values):
    avg = mean(values)
    return sum((x - avg) ** 2 for x in values) / len(values)

def gaussian(x, avg, var):
    var = max(var, 1e-9)
    return (1 / math.sqrt(2 * math.pi * var)) * math.exp(
        -((x - avg) ** 2) / (2 * var)
    )

def predict(sample):
    scores = {}

    for cls, rows in classes.items():
        score = math.log(len(rows) / len(data))

        for feature in FEATURES:
            values = [float(row[feature]) for row in rows]
            score += math.log(gaussian(
                float(sample[feature]), mean(values), variance(values)
            ))

        scores[cls] = score

    return max(scores, key=scores.get)

patient = {feature: data[0][feature] for feature in FEATURES}
print("Predicted HeartDisease:", predict(patient))
```

7. K-Means Clustering

Overall purpose: Groups numeric data into K clusters without using class labels.

- `random.sample(data, K)` chooses K different data points as the initial centroids.
- `distance(a, b)` calculates Euclidean distance between two points.
- `range(100)` limits the algorithm to at most 100 update rounds.
- `clusters = [[] for _ in range(K)]` creates K empty lists for the current groups.
- `min(range(K), key=...)` finds the index of the nearest centroid.
- `append(point)` places the data point in that centroid's cluster.
- The new centroid is calculated by taking the mean of each column in its cluster.
- If a cluster becomes empty, its old centroid is retained.
- If the centroids do not change, `break` stops the loop early.

Important: data.csv must contain numeric values only, with no header row. Set K to the required number of clusters.

Code:

```
import csv
import math
import random

with open("data.csv", "r") as file:
    data = [[float(value) for value in row]
            for row in csv.reader(file) if row]
```

```
K = 3
random.seed(1)
centroids = random.sample(data, K)

def distance(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

for _ in range(100):
    clusters = [[] for _ in range(K)]

    for point in data:
        nearest = min(range(K),
                      key=lambda i: distance(point, centroids[i]))
        clusters[nearest].append(point)

    new_centroids = []
    for i, cluster in enumerate(clusters):
        if cluster:
            new_centroids.append([
                sum(point[j] for point in cluster) / len(cluster)
                for j in range(len(data[0]))
            ])
        else:
            new_centroids.append(centroids[i])

    if new_centroids == centroids:
        break
    centroids = new_centroids

for i, cluster in enumerate(clusters):
    print("Cluster", i + 1, ":", cluster)
    print("Centroid:", centroids[i])
```

8. K-Nearest Neighbours (KNN)

Overall purpose: Predicts a label by finding the K closest labelled training examples.

- `X` stores all feature columns as floating-point numbers; `y` stores the last column, the class label.
- `zip(a, b)` pairs corresponding feature values from two points.
- `distance` calculates Euclidean distance between the test point and one training point.
- `distances` stores pairs containing distance and the corresponding known label.
- `sorted(..., key=lambda item: item[0])` sorts neighbours from nearest to farthest.
- `[:k]` selects only the first K neighbours.
- `Counter(...).most_common(1)[0][0]` finds the most frequent label among those neighbours.
- The sample is predicted using K=3 in the final print statement.

Important: knn_data.csv must have a header row, numeric feature columns, and the class label in the last column.

Code:

```
import csv
import math
from collections import Counter

with open("knn_data.csv", "r") as file:
    data = list(csv.reader(file))

X = [[float(value) for value in row[:-1]] for row in data[1:]]
y = [row[-1] for row in data[1:]]

def distance(a, b):
    return math.sqrt(sum((x - z) ** 2 for x, z in zip(a, b)))

def predict(sample, k=3):
    distances = [
        (distance(sample, X[i]), y[i])
        for i in range(len(X))
    ]
    neighbours = sorted(distances, key=lambda item: item[0])[:k]
    return Counter(label for _, label in neighbours).most_common(1)[0][0]

test_sample = X[0]
print("Predicted Class:", predict(test_sample, k=3))
```

9. Locally Weighted Regression

Overall purpose: Predicts a numerical value by fitting a weighted linear model around each query point.

- `numpy` provides arrays and matrix operations; `matplotlib.pyplot` is used to draw the result.
- `np.array(...)` converts the CSV rows into a numerical array.
- `data[:, 0]` selects every row from the first column; `data[:, 1]` selects the second column.
- `np.exp(...)` calculates the exponential weights. Points closer to the query receive larger weights.
- `np.diag(weights)` turns the one-dimensional weights into a diagonal matrix.
- `np.column_stack((np.ones(len(X)), X))` creates the design matrix with a column of ones for the intercept and a column for X.
- `np.linalg.pinv(...)` calculates the pseudoinverse, helping solve the weighted least-squares equation.
- `np.linspace(...)` creates 100 evenly spaced query points for drawing a smooth curve.
- `plt.scatter` draws the original data; `plt.plot` draws the predictions.

Important: data.csv must have a header row and two numeric columns: X and Y. This program needs NumPy and Matplotlib.

Code:

```
import csv
import numpy as np
import matplotlib.pyplot as plt

with open("data.csv", "r") as file:
    rows = list(csv.reader(file))

data = np.array([[float(value) for value in row] for row in rows[1:]])
X = data[:, 0]
y = data[:, 1]

def predict(query, tau=0.5):
    weights = np.exp(-((X - query) ** 2) / (2 * tau ** 2))
    W = np.diag(weights)

    X_design = np.column_stack((np.ones(len(X)), X))
    theta = np.linalg.pinv(X_design.T @ W @ X_design) @ (
        X_design.T @ W @ y
    )

    return np.array([1, query]) @ theta

test_points = np.linspace(min(X), max(X), 100)
predictions = [predict(point) for point in test_points]

plt.scatter(X, y, label="Training Data")
plt.plot(test_points, predictions, label="LWR Prediction")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Locally Weighted Regression")
plt.legend()
plt.show()
```

Python building blocks used in these programs

Variables: A variable is a name that refers to a value, such as `K = 3`.

Lists: Lists store ordered values. Indexing starts at 0; `row[-1]` means the last item.

Loops: `for` repeats a block for each item. Indentation shows which statements belong to the loop.

Conditions: `if`, `elif` and `else` choose which block runs.

Functions: `def` defines reusable code. `return` sends a result back to the caller.

Dictionaries: A dictionary stores key-value pairs. `row['Play']` accesses the value stored under the key Play.

Comprehensions: An expression such as `[x for x in data]` builds a new list by processing each item.

Important symbols: == checks equality, `!=` checks inequality, = assigns a value, `:` begins an indented block, and # begins a comment.

File reminders: Keep the CSV file in the folder from which you run the Python program. Column headings and file names must match the code exactly.